

TP2

Le robot collecteur prend vie

Sprites Animations Tilemap Parallaxe et retour visuel

Durée	Type	Point de départ	Remise
90 minutes	Individuel	Dépôt du TP1	GitHub

Mise en situation

Le Robot Collecteur du TP1 fonctionne, mais son environnement est encore composé de formes simples. Vous devez améliorer sa présentation sans briser son déplacement, la collecte des batteries, la zone interdite ni la porte de sortie.

Objectif du TP2

Importer et configurer des ressources 2D, créer les animations Idle et Move, construire une Tilemap, produire une profondeur visuelle avec trois couches de parallaxe et afficher un retour visuel lors d'un contact avec la zone interdite.

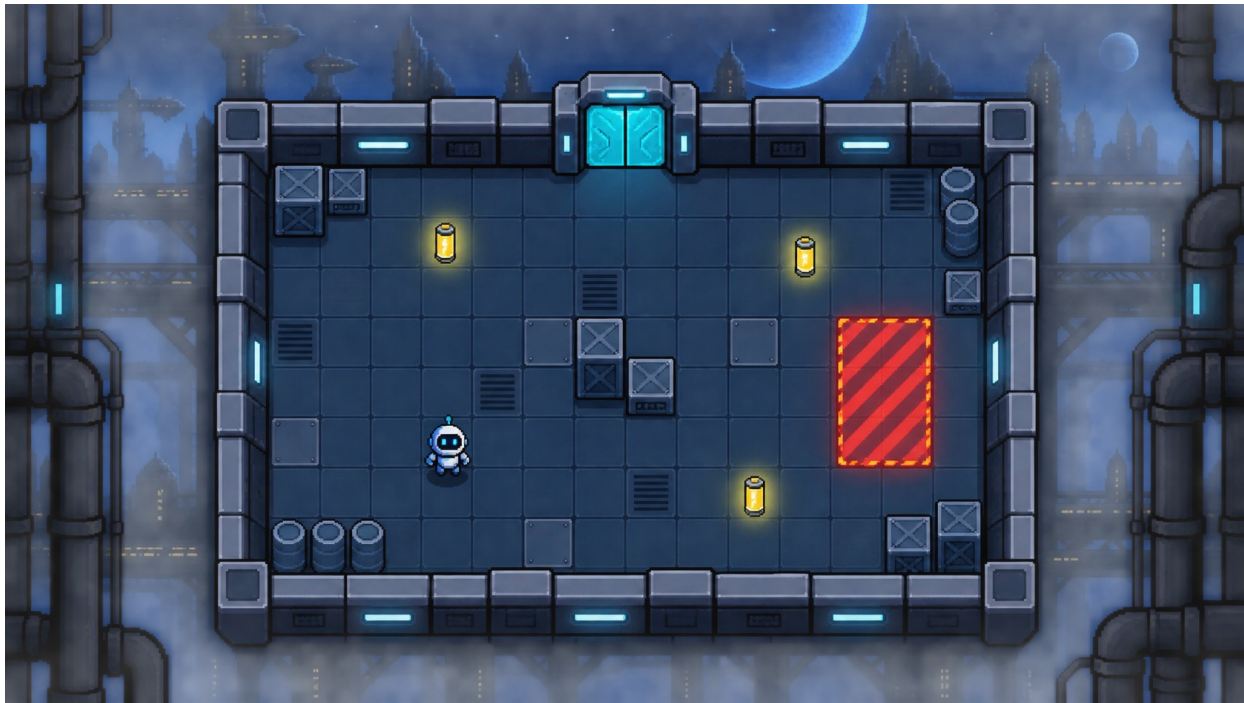
Contrainte essentielle

Le TP2 doit être réalisé dans le même dépôt GitHub que le TP1. Chaque fonctionnalité doit avoir son propre commit et son propre push.

Résultat attendu

- ❑ Les formes du robot, des batteries et de la porte sont remplacées par des sprites.
- ❑ Le robot joue Idle lorsqu'il est immobile et Move lorsqu'il se déplace.
- ❑ La scène contient trois Tilemaps distinctes : Sol, Murs et Decoration.
- ❑ Trois couches de fond se déplacent à des vitesses apparentes différentes.
- ❑ Le robot et l'écran produisent un retour visuel bref dans la zone interdite.
- ❑ La collecte, le retour au départ et la porte du TP1 fonctionnent toujours.
- ❑ La Console ne contient aucune erreur rouge.

Vue de référence à reproduire



La disposition exacte peut varier. La scène doit toutefois contenir un robot, trois batteries accessibles, une porte de sortie, une zone interdite rouge, des murs, un sol construit avec la Tilemap et trois couches de parallaxe.

Ressources fournies

Fichier	Utilisation
Vue_A_Reproduire.png	Modèle visuel du résultat attendu
SpriteSheet_Robot_Batterie_Porte_4x4.png	Robot, batteries et porte
Tileset_Spatial_4x4.png	Sol, murs, zone interdite et décoration
Fond_Lointain.png / Fond_Usine.png / Fond_PremierPlan.png	Trois couches de parallaxe

Organisation des 90 minutes

Mission	Travail	Durée	Commit attendu
1	Préparer une copie de la scène	5 min	Préparer le TP2
2	Remplacer les formes par des sprites	10 min	Ajouter les sprites
3	Créer Idle et Move	15 min	Créer les animations
4	Relier le script à l'Animator	10 min	Relier le mouvement
5	Construire la Tilemap	15 min	Construire le niveau
6	Ajouter la parallaxe	15 min	Ajouter la profondeur
7	Ajouter le retour visuel	15 min	Ajouter l'effet de danger

8	Tester et remettre	5 min	Corriger et finaliser
---	--------------------	-------	-----------------------

Mission 1 Préparer le projet 5 minutes

Ouvrez le dépôt du TP1 et vérifiez qu'il est à jour. Dupliquez la scène fonctionnelle et nommez la copie Scene_Sprites. Créez les dossiers Animations, Sprites, SpriteSheets, Tilemap et Tiles dans Assets.

- ☐ Le robot se déplace.
- ☐ Les trois batteries peuvent être collectées.
- ☐ La zone interdite renvoie le robot au PointDepart.
- ☐ La porte apparaît et termine la mission.

```
git add .
git commit -m "Préparer la scène du TP2"
git push
```

Mission 2 Ajouter les sprites 10 minutes

Importez SpriteSheet_Robot_Batterie_Porte_4x4.png. Choisissez Texture Type = Sprite 2D and UI, Sprite Mode = Multiple et Pixels Per Unit ≈ 300 . Dans Sprite Editor, choisissez Slice > Grid by Cell Count, puis indiquez 4 colonnes et 4 lignes, avec Pivot = Center. Cliquez sur Slice, Apply, puis fermez l'éditeur.

Ne supprimez aucun comportement

Conservez Rigidbody2D, Collider2D, Animator et les scripts. Le Sprite Renderer contrôle l'apparence; les autres composants contrôlent le comportement.

Ligne de la feuille	Contenu	Utilisation
1	Robot Idle	Animation du robot immobile
2	Robot Move	Animation du robot en déplacement
3	Batteries	Trois instances visibles dans la scène
4	Porte	Porte cachée au départ, puis visible après la collecte

Affectez une image appropriée au Sprite Renderer du robot, au prefab Batterie et à PorteSortie. Ajustez l'échelle des objets au besoin, sans modifier leurs comportements.

```
git add .
git commit -m "Remplacer les formes par des sprites"
git push
```

Mission 3 Créer Idle et Move 15 minutes

Dans la feuille déjà découpée, sélectionnez les quatre images de la première ligne et créez Robot_Idle.anim. Sélectionnez ensuite les quatre images de la deuxième ligne et créez Robot_Move.anim. Enregistrez les clips dans Assets/Animations.

Clip	Frames	Loop Time	Samples	État
RobotAnimation.anim	4 frames Idle	Activé	4 à 6	Robot_Idle par défaut
Robot_Move.anim	4 frames Move	Activé	8 à 10	Robot_Move

Dans l'Animator, créez un paramètre Bool nommé exactement EnMouvement, puis configurez les transitions suivantes :

Transition	Has Exit Time	Duration	Condition
Robot_Idle vers Robot_Move	Désactivé	0 ou 0,05	EnMouvement = true
Robot_Move vers Robot_Idle	Désactivé	0 ou 0,05	EnMouvement = false

```
git add .
git commit -m "Créer les animations Idle et Move"
git push
```

Mission 4 Relier le script à l'Animator 10 minutes

Adaptez MouvementRobot.cs afin de récupérer l'Animator et de lui transmettre l'état du déplacement. Le code de collecte ne doit pas être déplacé dans ce script.

BANQUE DE LIGNES À REPLACER

```
animator.SetBool("EnMouvement", direction.sqrMagnitude > 0.01f);
private Animator animator;
animator = GetComponent<Animator>();
[RequireComponent(typeof(Animator))]
```

Placez chaque ligne au bon endroit : attribut de classe, variable membre, Awake ou Update. Le paramètre EnMouvement doit recevoir true quand direction n'est pas nulle et false lorsque le robot est immobile.

Compatibilité avec le TP1

Conservez votre système d'entrée actuel. Il n'est pas nécessaire de remplacer tout le script si le déplacement fonctionne déjà; ajoutez seulement la gestion de l'Animator.

- ☐ Sans touche : Robot_Idle est actif.
- ☐ Avec WASD ou les flèches : Robot_Move est actif.
- ☐ Le passage entre les deux états est immédiat.
- ☐ La diagonale n'est pas plus rapide.

```
git add .
git commit -m "Relier le mouvement à l'Animator"
git push
```

Mission 5 Construire la Tilemap 15 minutes

Importez Tileset_Spatial_4x4.png avec Sprite Mode = Multiple et Pixels Per Unit ≈ 314 . Dans Sprite Editor, choisissez Slice > Grid by Cell Count, puis indiquez 4 colonnes et 4 lignes, avec Pivot = Center. Cliquez sur Slice et Apply. Créez ensuite une Grid contenant trois Tilemaps nommées Sol, Murs et Decoration.

```
Grid
  Sol
  Murs
  Decoration
```

Créez RobotPalette dans Window > 2D > Tile Palette et enregistrez les fichiers Tile dans Assets/Tiles. Peignez une petite zone jouable inspirée de la vue fournie : un sol, un contour de murs, une zone interdite rouge et au moins deux éléments décoratifs.

Collisions

Ajoutez Tilemap Collider 2D uniquement à Murs. Sol et Decoration ne doivent pas bloquer le robot.

- ☐ Le robot ne traverse aucun mur.
- ☐ La décoration ne bloque jamais le robot.

- ☐ Les batteries restent accessibles.
- ☐ La zone interdite et la porte restent utilisables.

```
git add .
git commit -m "Construire le niveau avec une Tilemap"
git push
```

Mission 6 Ajouter la parallaxe 15 minutes

Importez Fond_Lointain.png, Fond_Usine.png et Fond_PremierPlan.png comme sprites Single. Utilisez PPU = 80, Filter Mode = Point no filter et Compression = None. Activez Alpha Is Transparency pour Usine et PremierPlan.

```
ArrierePlan
  Fond_Lointain
  Fond_Usine
  Fond_PremierPlan
```

Ajoutez le script fourni EffetParallaxe.cs aux trois couches, puis affectez Main Camera au champ Camera Cible. Configurez les valeurs suivantes :

Couche	Horizontal	Vertical	Vitesse automatique
Fond_Lointain	0,92	0,95	X 0,005 Y 0
Fond_Usine	0,75	0,85	X 0,01 Y 0
Fond_PremierPlan	0,40	0,60	X 0 Y 0

Ordre d affichage

Valeurs conseillées : Fond_Lointain = -30, Fond_Usine = -20 et Fond_PremierPlan = -5. Ajustez le premier plan s'il cache le robot.

- ☐ Le ciel bouge très lentement.
- ☐ L'usine se déplace plus rapidement que le ciel.
- ☐ Le premier plan accentue la profondeur sans cacher le jeu.

```
git add .
git commit -m "Ajouter un effet de parallaxe"
git push
```

Mission 7 Ajouter le retour visuel 15 minutes

Dans le Canvas, créez UI > Image et nommez l'objet FlashDegat. Étirez ses ancrs sur tout l'écran, choisissez une couleur rouge et désactivez Raycast Target. Ajoutez un Canvas Group avec Alpha = 0, Interactable désactivé et Blocks Raycasts désactivé.

Ajoutez le script fourni EffetDegatsJoueur.cs au Robot. Affectez le Sprite Renderer du robot à Rendu Robot et le Canvas Group de FlashDegat à Flash Ecran.

Adaptez ZoneInterdite.cs en ajoutant les deux lignes suivantes aux endroits appropriés :

```
[SerializeField] private EffetDegatsJoueur effetDegatsJoueur;

effetDegatsJoueur?.JouerEffetDegat();
```

L'appel à JouerEffetDegat doit se produire après les validations du tag Player et de PointDepart, mais avant le retour du robot au point de départ.

Harmonie avec le TP1

Cette version ajoute un retour visuel à la ZoneInterdite sans introduire un système de vies. Le comportement évalué reste : effet bref, puis retour au PointDepart.

- ☐ Le robot clignote brièvement en rouge.
- ☐ Le flash apparaît puis redevient transparent.
- ☐ Le flash ne bloque aucun élément d'interface.
- ☐ Le robot revient au PointDepart.

```
git add .
git commit -m "Ajouter le retour visuel de la zone interdite"
git push
```

Mission 8 Validation et remise 5 minutes

Test	Action	Résultat attendu
A	Ne toucher à aucune commande	Idle joue en boucle
B	Déplacer le robot	Move démarre immédiatement
C	Longer les murs et la décoration	Murs bloquants; décoration traversable
D	Collecter les trois batteries	Compteur et porte fonctionnels
E	Déplacer la caméra	Trois vitesses de parallaxe visibles
F	Entrer dans la zone interdite	Effet visuel puis retour au PointDepart
G	Toucher la porte active	Fin de mission du TP1 conservée
H	Consulter la Console	Aucune erreur rouge

Effectuez un dernier push uniquement si une correction a été apportée. Vérifiez l'historique du dépôt avant de remettre son URL.

Livrables

- ☐ L'URL du même dépôt GitHub que le TP1.
- ☐ La scène Scene_Sprites.unity.
- ☐ Les ressources organisées dans les dossiers demandés.
- ☐ Les scripts EffetParallaxe.cs et EffetDegatsJoueur.cs intégrés à la scène.
- ☐ Au moins sept commits distincts correspondant aux fonctionnalités du TP2.
- ☐ L'enseignant demeure ajouté comme collaborateur.

Barème 10 points

Critère	Éléments observés	Points
GitHub	Même dépôt; commits et push séparés par fonctionnalité	0,5
Sprites	Importation correcte et remplacement sans briser les composants	1
Animations	Idle, Move, paramètre et transitions réactives	1,5
Script et Animator	Référence Animator et mise à jour de EnMouvement	1
Tilemap	Sol, Murs et Decoration correctement séparés	1,5
Parallaxe	Trois couches, réglages distincts et affichage correct	1,5
Retour visuel	Clignotement, flash et appel depuis ZoneInterdite	1,5
Conservation	Collisions et fonctionnalités du TP1 intactes	0,5
Question	Réponse claire et correcte	1

Question de compréhension 1 point

Pourquoi utilise-t-on des valeurs de suivi différentes pour les trois couches de parallaxe?

Réponse : Pour donner la sensation que chaque fond bouge de façon indépendante, cela veut dire que le "ciel" (fond_Lointain) bouge indépendamment des "usines" (fond_Usine) dans ce projet.

Avant de remettre

Ouvrez la scène depuis le dossier Assets, jouez une partie complète et consultez la Console. Une amélioration visuelle ne doit jamais supprimer une fonctionnalité déjà validée.